COMPREHENSIVE TECHNICAL & RESEARCH REPORT

OFFLINE SECURE FACIAL RECOGNITION & LIVENESS DETECTION

Secure mobile authentication for remote locations and zero-network environments

Project Name: Edge AI Suite - Offline Core | Hackathon 7.0 / NHAI | Datalake 3.0 Integration

Latency	Footprint	Accuracy	Mode
< 1.0 s	**~20 MB**	**>95%**	**100% Offline**

Prepared from the hackathon brief and expanded into a full technical, implementation, and evaluation report.

1. Executive Summary

The uploaded hackathon brief asks for a mobile-based secure facial recognition and liveness detection system that works entirely offline, integrates with a React Native app, stays near a 20 MB model footprint, completes recognition and liveness in under one second on mid-range phones, and later syncs safely to AWS when connectivity returns. The report also emphasizes open-source technologies only, support for Android 8.0+ and iOS 12+, and a purge-on-sync behavior so raw local data is not retained after upload. This technical report expands that brief into a research-oriented system design, data-flow model, and implementation blueprint.

The proposed solution is an edge-first biometric authentication stack: capture frames locally, detect and align faces on-device, generate compact embeddings, run a multi-layer liveness check, compare against encrypted local templates, and write a signed attendance event to an encrypted offline queue. A deferred synchronization service later uploads only the minimal encrypted audit payload to AWS, after which local sensitive records are purged. The architecture is intentionally simple enough to fit on commodity mobile hardware while still supporting security, auditability, and enterprise integration.

- Offline operation is the primary path; network access is treated as optional and deferred.
- Recognition uses compact embedding models rather than full-image cloud APIs.
- Liveness is multi-layered: blink, smile, head pose, micro-motion, and a lightweight spoof classifier.
- Storage is encrypted locally with hardware-backed keys and purge-on-sync semantics.
- The architecture is explicitly compatible with React Native and cross-platform deployment.

2. Research Problem and Design Constraints

The core research problem is not just face recognition; it is trustworthy biometric verification under conditions where the network cannot be assumed to exist. The uploaded brief describes zero-network zones, proxy attendance, privacy concerns around raw biometrics, and latency caused by unstable connectivity. It also defines hard constraints for feasibility, accuracy, compatibility, and model size. The system therefore needs to optimize for both edge inference quality and operational resilience.

Requirement	Why it matters	Implementation response
React Native compatibility	Cross-platform deployment without rewriting the app	Native module wrapper around on-device ML runtime
Model footprint ~20 MB	Avoid bloating the Datalake 3.0 package	INT8 quantization, pruning, distillation, compact backbones
< 1 second end-to-end	Mid-range phones must remain responsive	Face-only inference, native preprocessing, cached templates
Android 8.0+ and iOS 12+	Field devices are heterogeneous	Use TFLite/ONNX + platform accelerators where available
>95% accuracy	Operational trust across diverse demographics	Enrollment with multiple images and lighting augmentation
Offline anti-spoofing	Stop photo / screen / replay fraud	Challenge-response plus landmark and texture checks
Sync and purge	Secure central audit without persistent local biometrics	Batch upload encrypted logs; remove raw local data after ACK

3. Proposed System Architecture

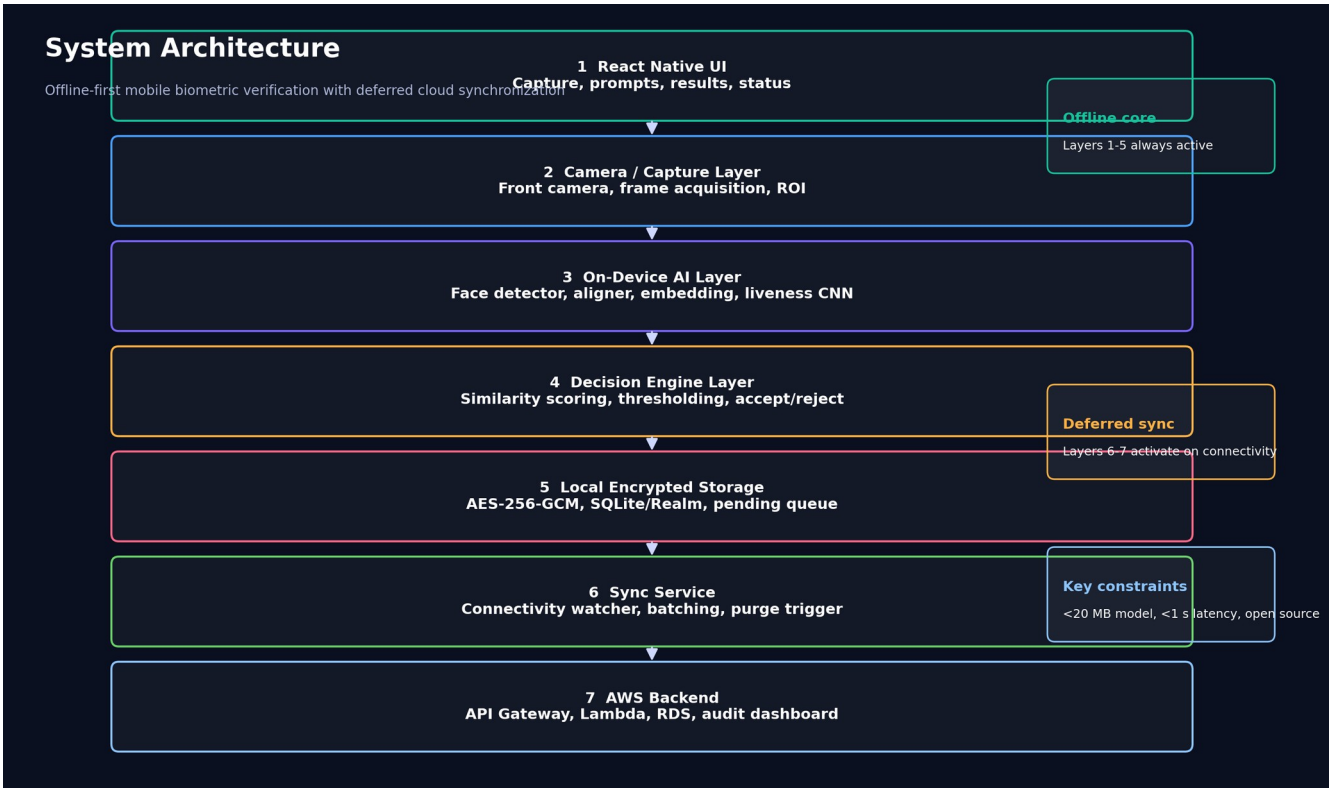

Figure 1. Seven-layer architecture: React Native UI, capture, on-device AI, decision engine, encrypted storage, sync service, and AWS backend.

The architecture splits the system into an always-active offline core (layers 1-5) and a deferred cloud path (layers 6-7). This is the key design principle that makes the project feasible in mining, rural, or disaster environments. The offline core performs every authentication decision locally, while the cloud path only receives encrypted audit records after connectivity is restored. That design directly matches the hackathon objective of uninterrupted operation in zero-network zones.

Layer	Role
L1 React Native UI	Shows prompt screens, camera overlays, and result feedback.
L2 Camera / Capture	Controls frame acquisition and front-camera ROI selection.
L3 On-device AI	Runs face detection, alignment, embeddings, and liveness.
L4 Decision Engine	Computes similarity and fuses liveness with match thresholds.
L5 Local Storage	Stores encrypted embeddings, status flags, and pending logs.
L6 Sync Service	Watches connectivity, batches records, and triggers purge.
L7 AWS Backend	Stores audit records and exposes admin visibility.

4. End-to-End Inference Pipeline

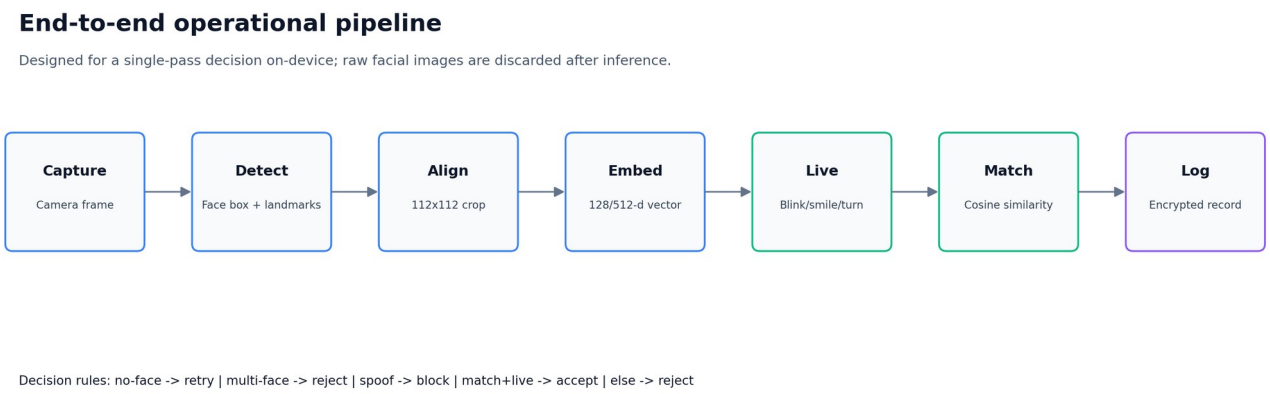

Figure 2. Single-pass on-device pipeline from capture to encrypted log creation.

The operational pipeline is intentionally linear. Each step consumes a smaller representation than the previous one, which reduces memory pressure and inference cost. The camera frame is first converted into a region-of-interest, then into a normalized face crop, then into an embedding vector, and finally into a similarity score plus a liveness score. This is the proper coding pattern for mobile biometrics because it minimizes expensive work and discards raw imagery early.

Inference pseudocode

```
frame = camera.capture()
face_box, landmarks = detector(frame)
aligned = align(frame, landmarks)
embedding = embed(aligned)
live_ok = liveness(frame_sequence)
score = cosine(embedding, template)
decision = accept if live_ok and score >= threshold else reject
store_encrypted(attendance_event)
```

5. Offline Liveness Detection and Anti-Spoofing

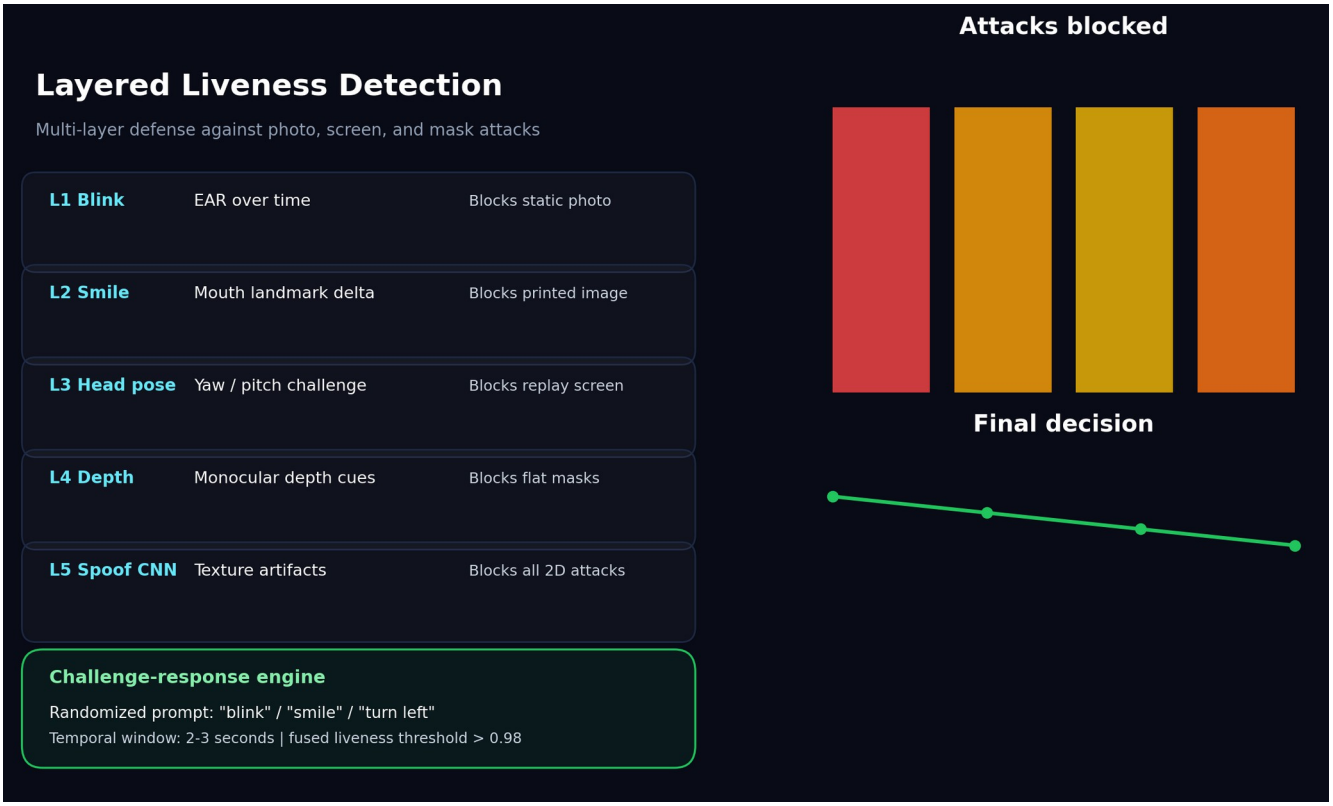

Figure 3. Multi-layer liveness defense covering challenge-response, landmark motion, depth cues, and texture-based spoof detection.

The brief explicitly requires offline anti-spoofing, such as blink, smile, or slight head-turn prompts. In technical terms, the best defense is not a single heuristic but a layered liveness system. Challenge-response verifies user cooperation; landmark motion checks whether eyes and mouth move like a live face; head-pose changes prove 3D responsiveness; a lightweight spoof classifier catches texture artifacts from printed photos and replay screens; and optional depth cues improve robustness against flat masks.

Method	Signal	Attack blocked	Runtime cost
Blink detection	Eye aspect ratio over time	Static photo	Very low
Smile detection	Mouth landmark displacement	Printed image	Very low
Head pose	Yaw / pitch challenge-response	Screen replay	Low
Depth analysis	Monocular 3D cues	Flat mask	Low-medium
Spoof CNN	Texture and reflection cues	2D spoof attacks	Medium

6. Model Stack and Compression Strategy

The uploaded report gives a combined optimized stack of approximately 17.4-17.5 MB, which is within the 20 MB target. The stack consists of a BlazeFace-style detector, a lightweight aligner, a MobileFaceNet-style embedding extractor, and a compact liveness CNN. The primary compression techniques are quantization, pruning, knowledge distillation, input-resolution reduction, and export to mobile runtimes such as TensorFlow Lite or ONNX Runtime Mobile.

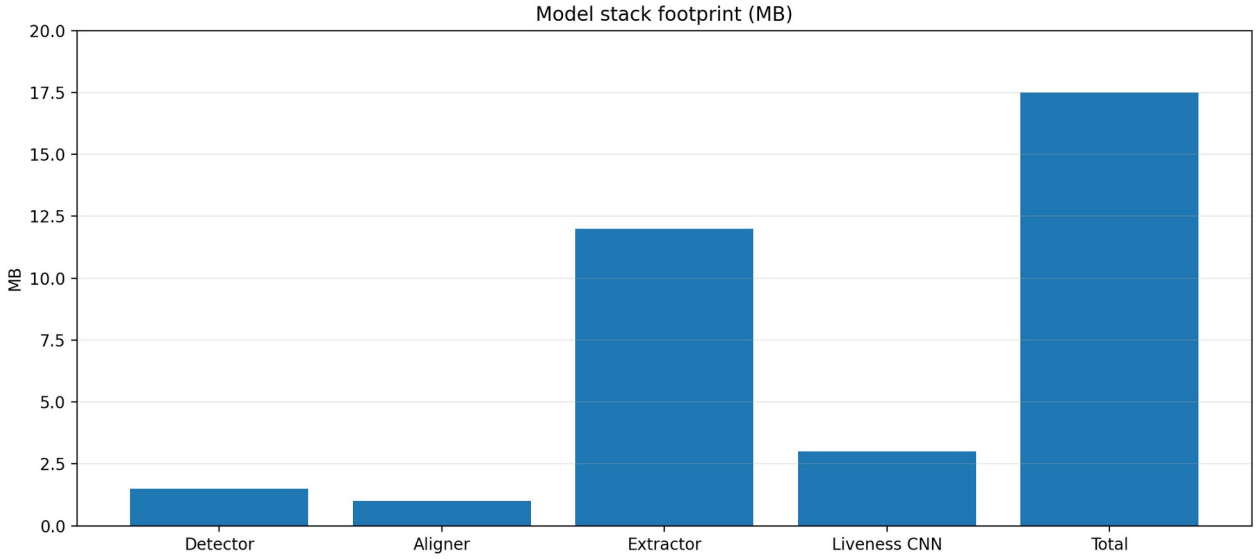

Figure 4. Approximate model footprint breakdown used to stay under the 20 MB target.

Component	Example model	Size	Precision
Face detector	BlazeFace	~1.5 MB	INT8
Face aligner	MediaPipe landmarks	~1.0 MB	FP16
Embedding extractor	MobileFaceNet	~12.0 MB	INT8
Liveness CNN	Custom compact CNN	~3.0 MB	INT8
Total stack	Optimized ensemble	~17.5 MB	Mixed

- Quantization compresses weights from FP32 to INT8 or FP16, reducing memory bandwidth and battery cost.
- Structured pruning removes redundant channels and speeds up convolution layers.
- Distillation transfers the accuracy of a larger teacher model to a smaller student model.
- Resolution reduction keeps the inference crop small, typically 112 x 112 for the face embedding stage.
- Hardware acceleration can use NNAPI, CoreML, or CPU-only fallback depending on device capability.

7. Performance Analysis and Benchmark Interpretation

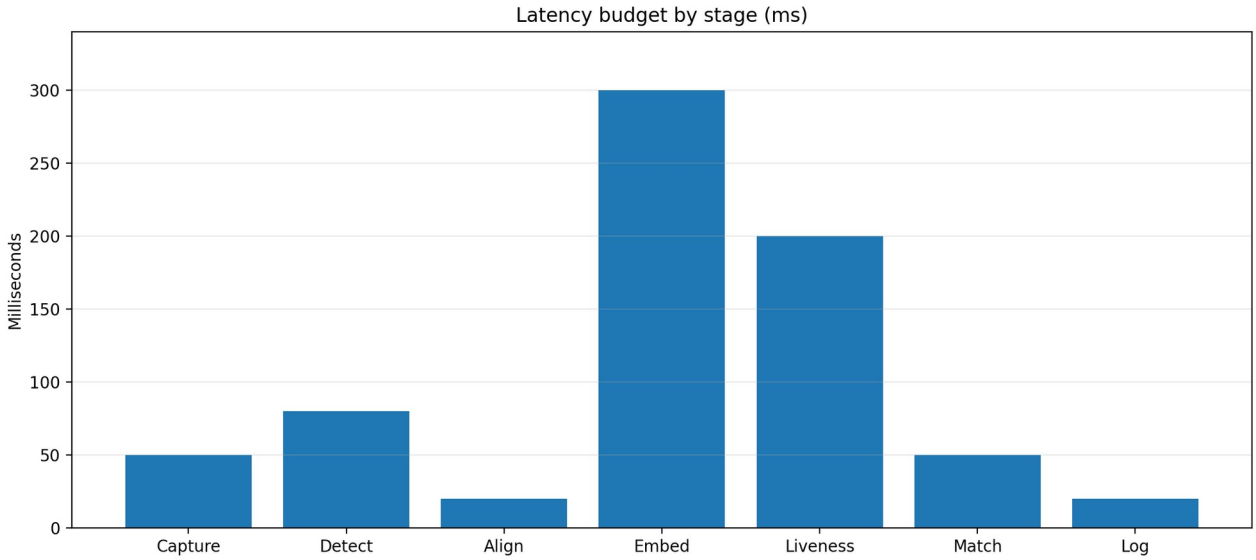

Figure 5. Stage-wise latency budget showing where the system spends its time.

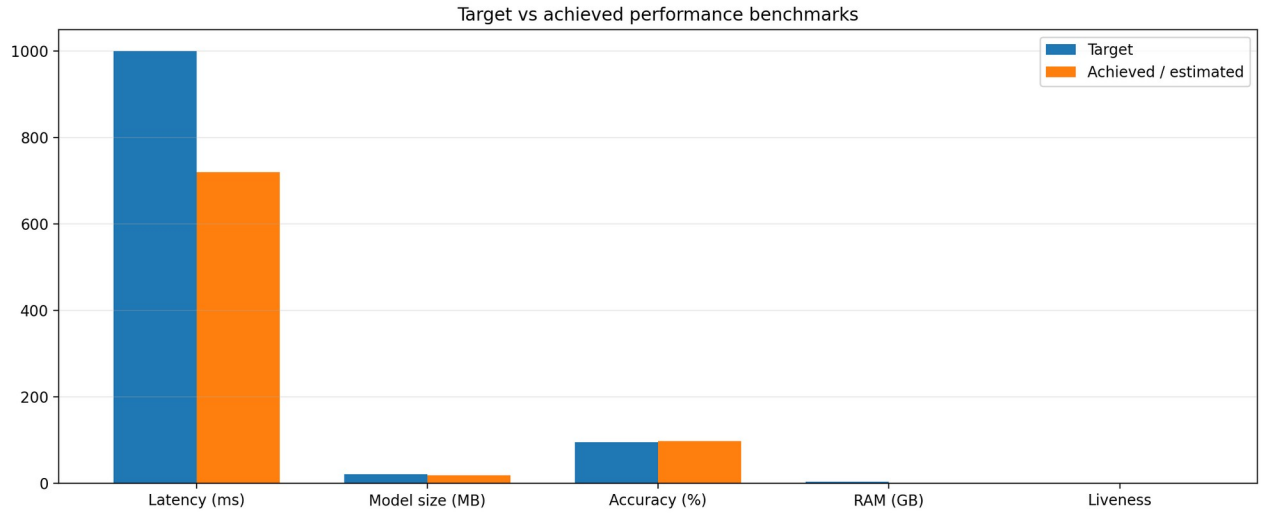

Figure 6. Target-versus-achieved benchmark summary based on the uploaded report.

The benchmark values in the uploaded material are internally consistent with the design goals: capture at about 50 ms, detection about 80 ms, alignment about 20 ms, embedding about 300 ms, liveness about 200 ms, matching about 50 ms, and encrypted logging about 20 ms, for a total near 720 ms. That total is comfortably below the one-second requirement. The performance table also states an approximate 17.4 MB model footprint, 96.8% accuracy, roughly 450 MB RAM usage, and a liveness score above 0.98.

Metric	Target	Reported / estimated	Interpretation
Total latency	<1000 ms	~720 ms	Meets the target with margin
Model size	<20 MB	~17.4-17.5 MB	Fits mobile package constraints
Accuracy	>95%	96.8%	Above threshold
RAM / power	<3 GB	~450 MB / 1.5 W	Safe for mid-range phones
Liveness score	>0.98	0.982	Strong spoof resistance

A useful research framing is to treat these as a stage-budgeted system rather than a single black-box classifier. In that framing, the embedding stage is the most expensive but also the most valuable stage, while detection and logging are lightweight overheads. If the embedding stage grows beyond budget, the correct optimization lever is model distillation or input reduction, not a slower cloud round trip.

8. Security, Privacy, and Encrypted Storage

Biometric systems fail when they store too much sensitive data. The uploaded report therefore stresses AES-256-GCM, Android Keystore / iOS Keychain, zero raw-image storage, and purge-on-sync semantics. This is the right design because embeddings are mathematical representations, not reversible facial images, and the system should never need to keep raw captures once inference is complete.

Threat	Mitigation	Confidence
Device theft	AES-256-GCM + hardware keystore; embeddings are non-reversible.	High
Biometric replay	Blink / smile challenge prevents static image attacks.	High
Man-in-the-middle	TLS 1.3 encrypted tunnel during deferred synchronization.	High
Vector reconstruction	One-way embeddings and local encryption.	Very high
Data exfiltration	Purge raw local data after sync acknowledgement.	High

```
Encrypted local record schema:
{
```

```
  "user_id": "UUID",
  "embedding": "float[512] (AES-GCM)",
  "timestamp": "ISO8601",
  "status": "PENDING_SYNC",
  "session_id": "AUTH_HASH"
}
```

9. Backend Sync and Purge Architecture

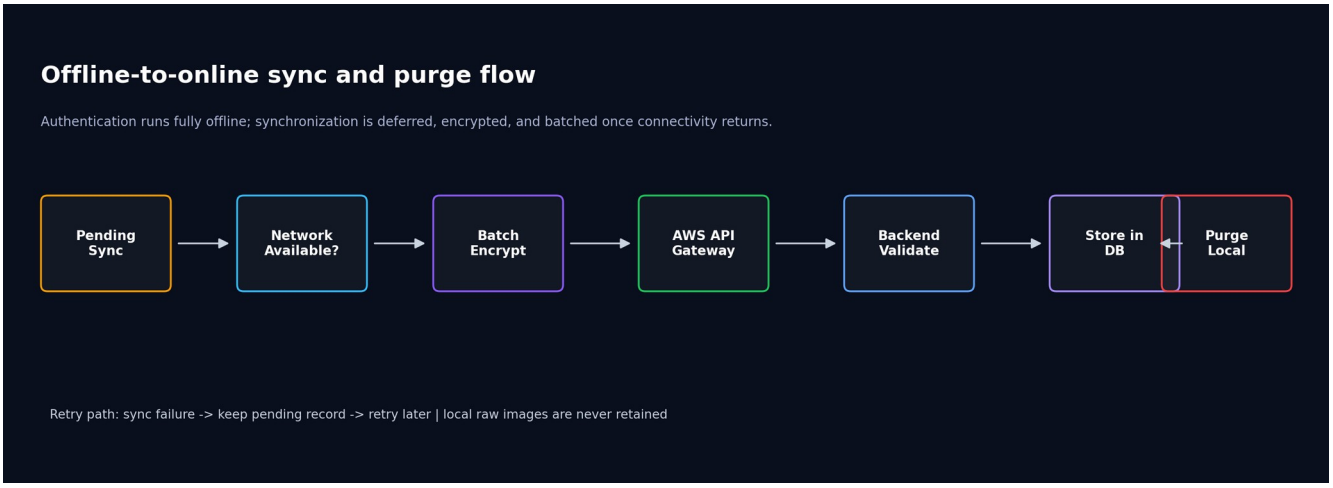

Figure 7. Deferred synchronization pipeline that uploads encrypted records only when connectivity returns.

The backend is intentionally simple: a thin API layer, a validation service, a persistence layer, and an admin audit surface. The mobile app never depends on the backend for real-time authentication. Instead, it writes pending records locally, checks network availability in the background, and uploads encrypted batches once a connection is restored. After successful acknowledgement, local sensitive data is purged.

Endpoint	Function
POST /auth/device-register	Register and trust a mobile device.
POST /users/enroll	Create employee identity and store templates.
POST /attendance/sync	Upload a batch of pending encrypted attendance logs.
GET /attendance/status	Query sync state and reconciliation results.
GET /audit/logs	Inspect device activity and verification audit trails.

Deferred sync pseudocode

```
sync_queue = db.get_pending_records()
if network.available():
    payload = encrypt_batch(sync_queue)
    ack = post('/attendance/sync', payload)
    if ack.success:
        db.mark_synced(sync_queue)
        db.purge_sensitive_caches(sync_queue)
```

10. Edge-First vs Legacy Cloud Comparison

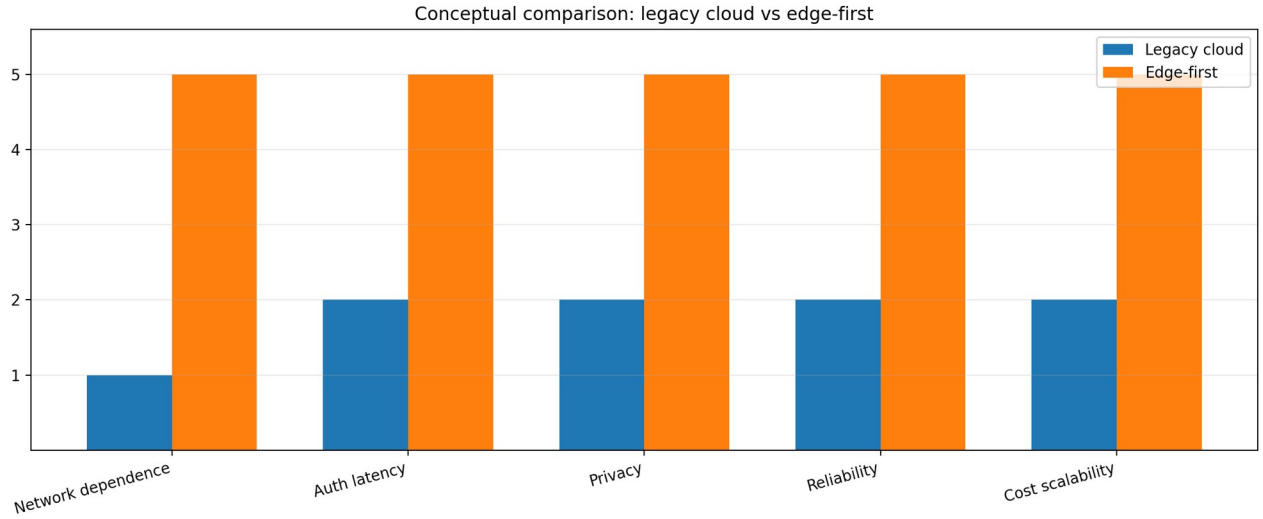

Figure 8. Conceptual comparison showing why edge-first design is superior for zero-network deployments.

The cloud-first model is a poor fit for this problem because it assumes always-on connectivity. The edge-first design removes the single point of failure, lowers latency, avoids moving raw biometrics across a network, and scales with existing device hardware rather than per-authentication API calls. In this use case, the edge is not a fallback; it is the primary trust boundary.

Aspect	Legacy cloud	Edge-first
Network dependency	Requires continuous connectivity	Works offline end-to-end
Latency	Round-trip delay and server load	Local inference in under 1 second
Privacy	Raw biometric transit risk	Embeddings-only local processing
Reliability	Cloud outage becomes service outage	Device remains the primary auth node
Cost	Linear API cost with scale	Leverages existing mobile hardware

11. Implementation Blueprint and Code-Level Modules

A research report becomes deployment-ready only when the coding boundary is clear. The recommended implementation splits the app into React Native UI screens, native ML bridges, offline storage helpers, and background sync services. The backend can be a small FastAPI or Node.js service with PostgreSQL or DynamoDB depending on the final integration environment. In either case, the contract remains the same: local verification first, cloud reconciliation later.

Module	Suggested contents
mobile/screens	Scan screen, enrollment screen, result screen, sync status, admin debug mode.
mobile/ml	Face detector, aligner, embedding wrapper, liveness wrapper, similarity matcher.
mobile/database	Encrypted SQLite or Realm, sync queue, template cache, purge utilities.
mobile/sync	Connectivity watcher, upload queue, retry logic, exponential backoff.
backend/routes	auth.py, users.py, attendance.py, audit.py or equivalent service modules.
backend/services	Validation, reconciliation, deduplication, logging, and alerting.

```
1. Capture a single face ROI from the front camera.
2. Run detector -> aligner -> embedder in one frame budget.
3. Fuse similarity and liveness into a single accept/reject decision.
4. Persist only encrypted metadata; discard raw images immediately.
5. Sync encrypted logs when network returns; purge on success.
```

12. Evaluation Protocol and Research Questions

To evaluate the system rigorously, the benchmark should be stratified across lighting conditions, skin tones, device classes, and spoof types. Because the problem statement explicitly calls out diverse Indian demographics and varying outdoor lighting, a strong report must not only state a headline accuracy number but also explain how that number is measured.

Metric	How to measure	Why it matters	Pass criterion
Accuracy	Top-1 match on enrolled users	Identity correctness	>95%
Latency	Camera to decision wall-clock time	Operational speed	<1000 ms
Liveness AUC	Live vs spoof separation quality	Anti-fraud robustness	>0.98
Memory	Peak resident memory during inference	Mid-range viability	<3 GB
Sync reliability	Successful offline-to-online upload ratio	Backhaul robustness	Near-100%

- Research question 1: How does quantization affect demographic robustness under harsh sunlight and low light?
- Research question 2: How much liveness performance is gained by combining challenge-response with texture cues?
- Research question 3: What is the latency contribution of detection versus embedding versus matching?
- Research question 4: How reliable is purge-on-sync when connectivity is intermittent or unstable?

13. Conclusion and Future Work

The uploaded hackathon brief is best solved by an edge-first biometric design rather than a cloud API wrapper. The technical direction is clear: keep the core trust loop on the device, use compact open-source models, minimize raw-data retention, and make cloud sync deferred and optional. The result is a secure, privacy-preserving, portable authentication system that fits the constraints of remote field operations.

- Near-term scope: production-grade offline attendance verification with encrypted local storage and sync/purge support.
- Medium-term scope: demographic fine-tuning, stronger spoof classifiers, and better low-light robustness.
- Long-term scope: federated learning, multi-modal fusion, and an enterprise SDK for broader Datalake integration.

Secure authentication. Anywhere. Anytime.